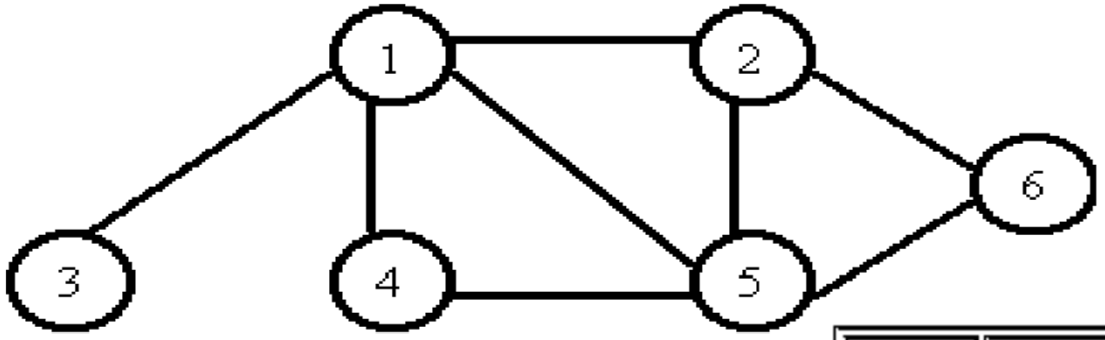
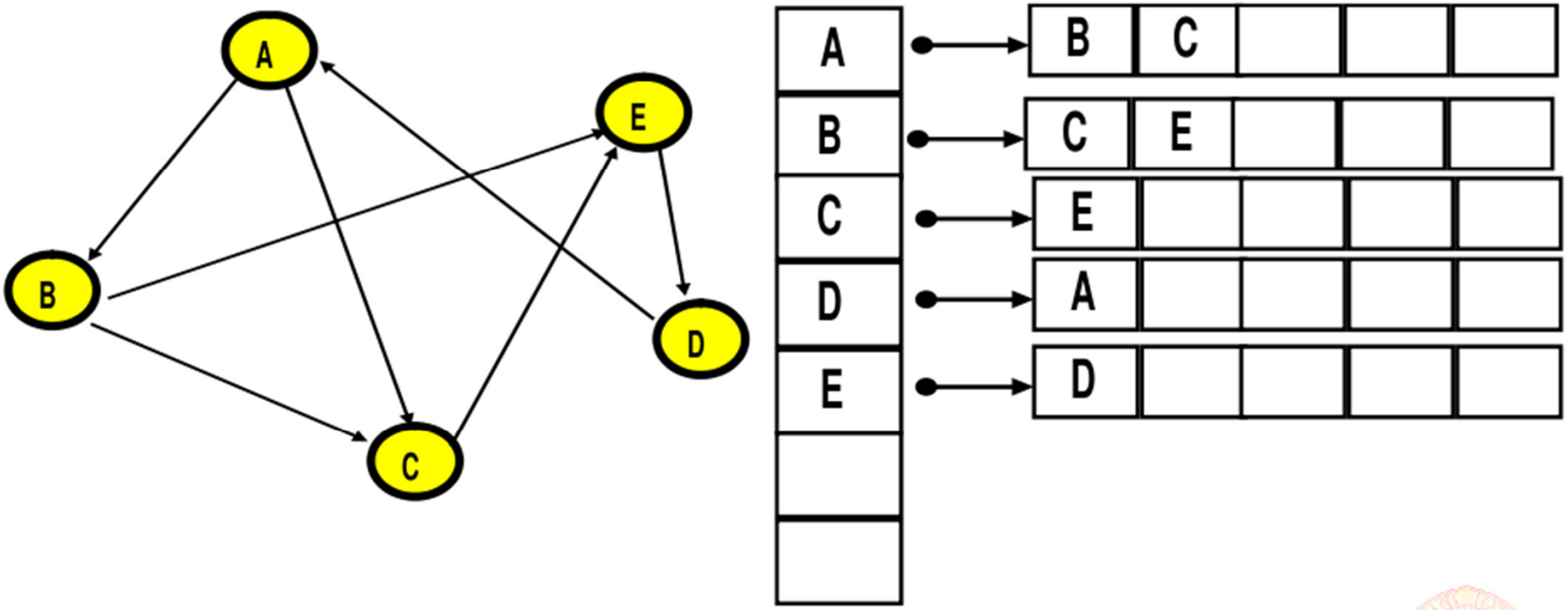
Aula 12 – Grafos

Matriz de Adjacência

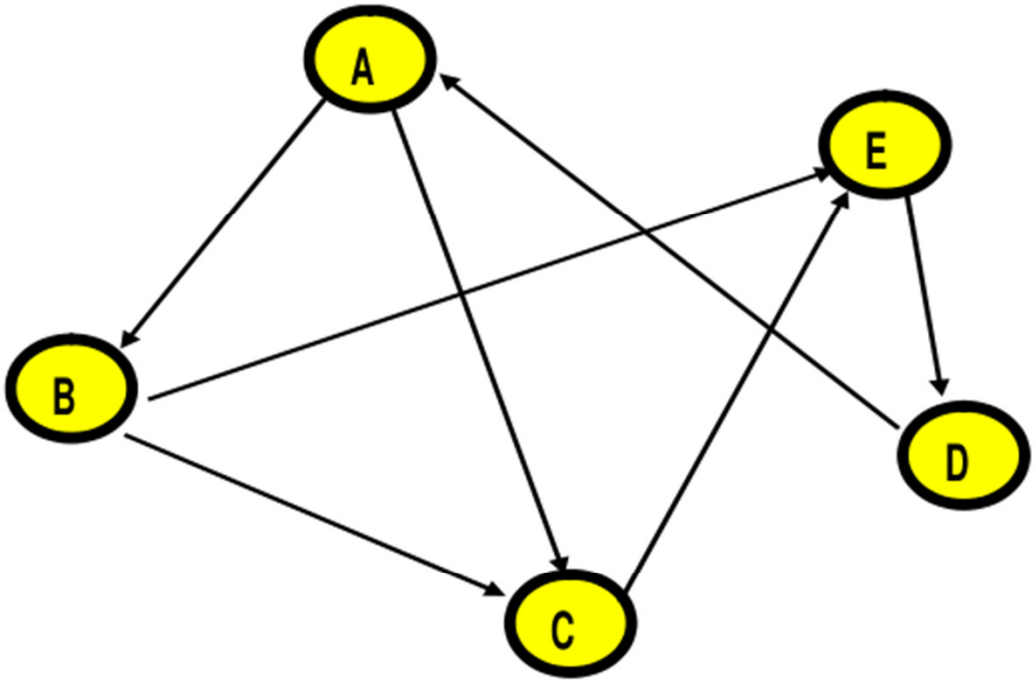


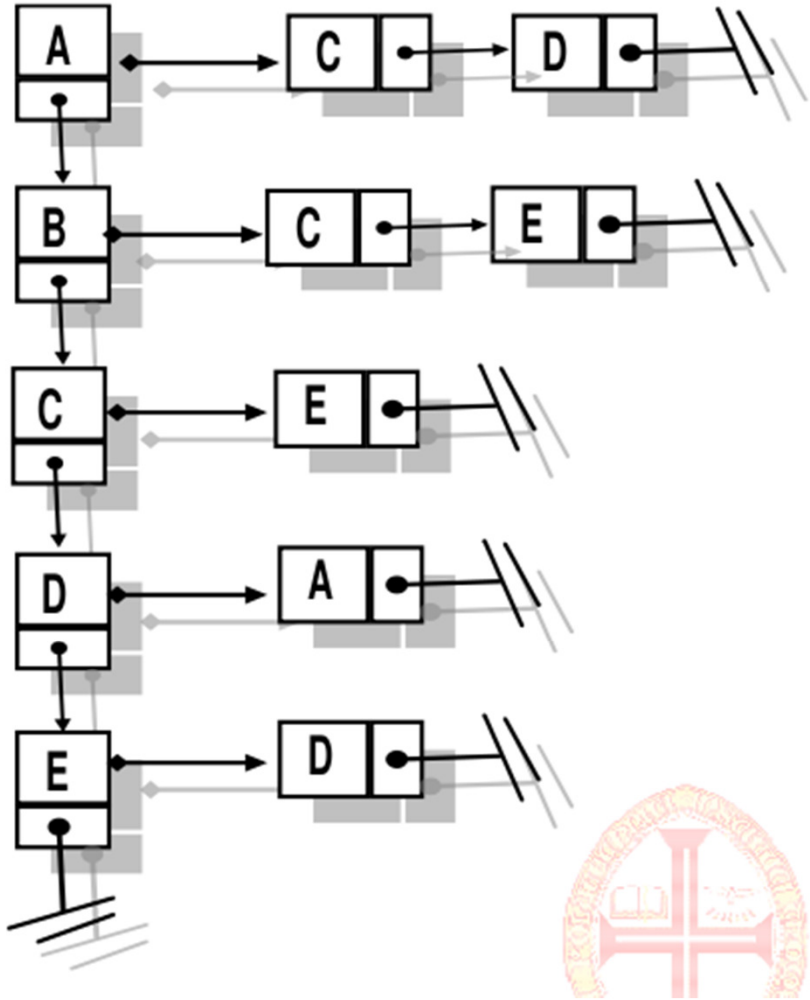
	1	2	3	4	5	6
1	0	1	1	1	1	0
2	1	0	0	0	1	1
3	1	0	0	0	0	0
4	1	0	0	0	1	0
5	1	1	0	1	0	1
6	0	1	0	0	1	0

Vectores

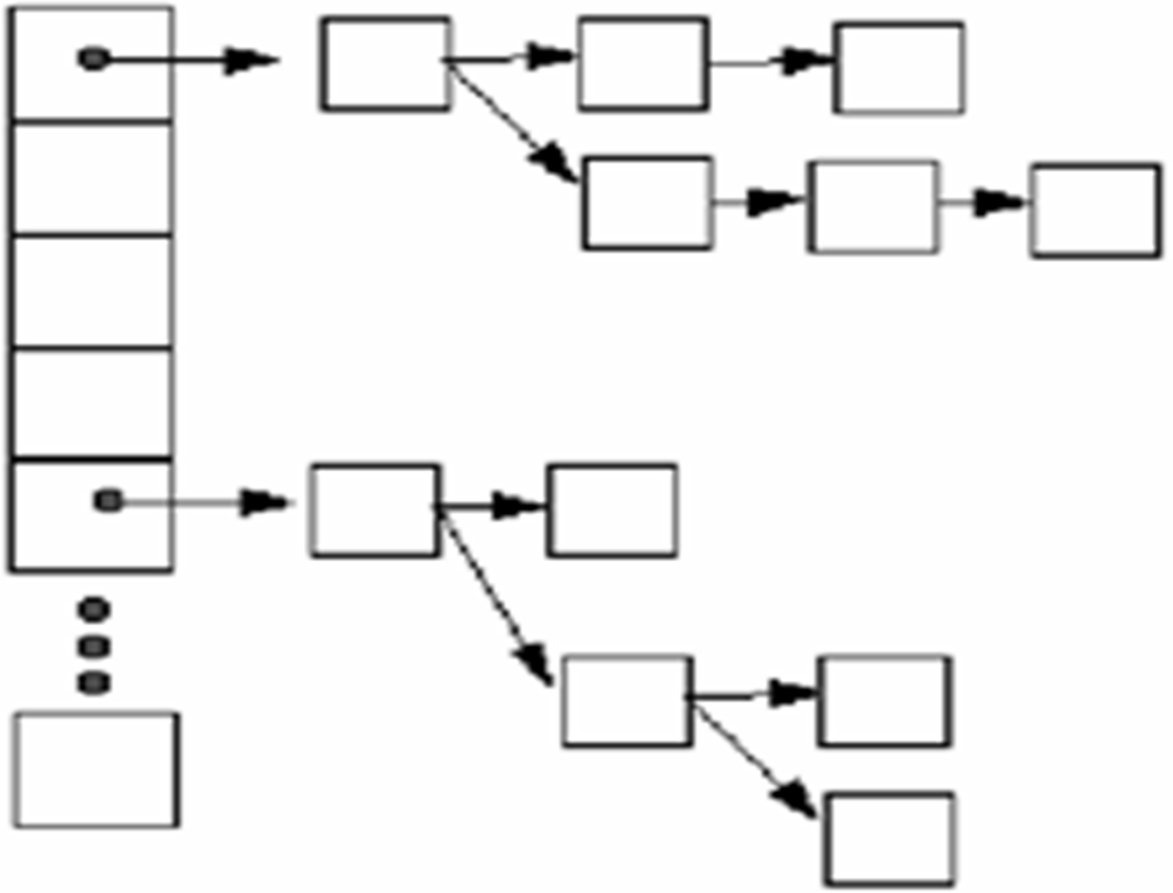


Listas ligadas





Arrays e Árvores

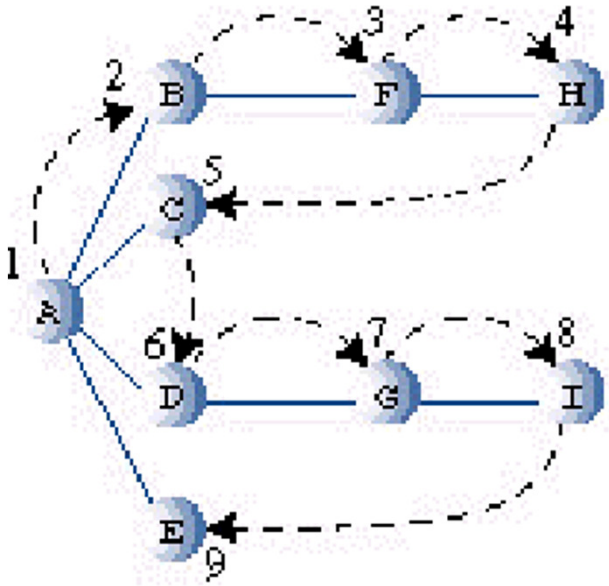


Percorrer o Grafo

- Dept First Search (DFS)
- Busca em profundidade

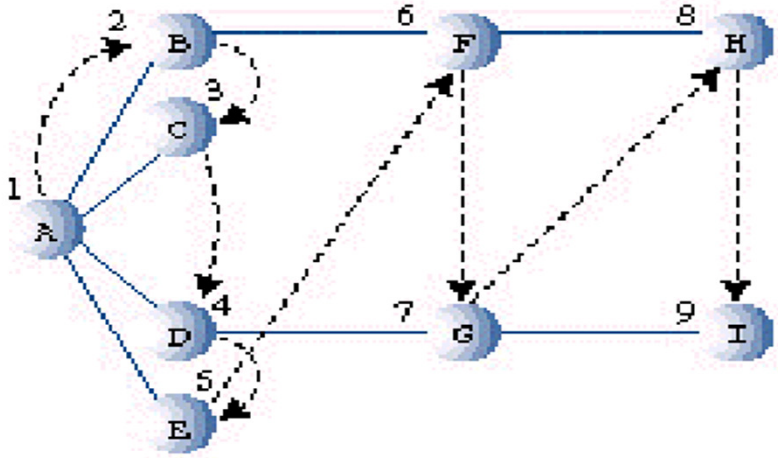
  ou em altura

```
funcao DFS(Inicio, Alvo)
    empilha (Pilha, Inicio)
    enquanto a Pilha não estiver vazia
        No = desempilha(Pilha)
        escreve (No)
        se No = Alvo devolve No
        para todos os Filhos do No
            se Filho não foi visitado
                empilha(Pilha, Filho)
```



Percorrer o Grafo

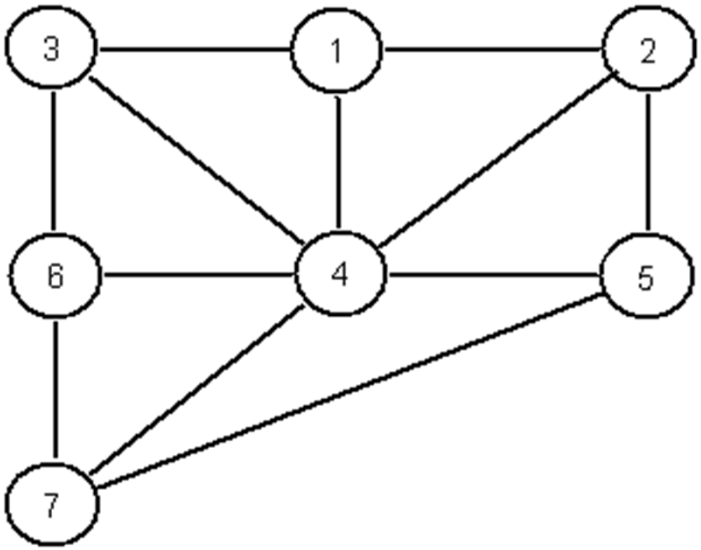
- Breath first Search (BFS)
- Busca em largura




```
funcao BFS (Inicio , Alvo)
    entra (Fila, Inicio)
    escreve (Inicio)
    enquanto a fila não estiver vazia
          No = sai (Fila)
    Se No = Alvo devolve No
          Para todos os Filhos do No
              Se Filho não foi visitado
                  escreve (Filho)
                  entra (Fila, Filho)
```

Percorrer o grafo

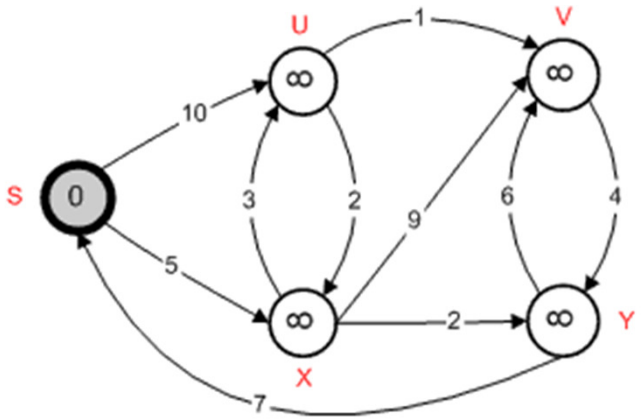
- BFS
- Largura
  - 3, 1, 4, 6, 2, 5, 7
- DFS
- Profundidade
  - 3, 1, 2, 5, 7, 4, 6



Aplicações

- Caminho mais curto
  - Algoritmo de Dijkstra
- Fluxo máximo
  - Algoritmo Guloso
  - Algoritmo de Ford-Fulkerson
- Árvore geradora mínima
  - Algoritmo de Prim
  - Algoritmo de Kruskal

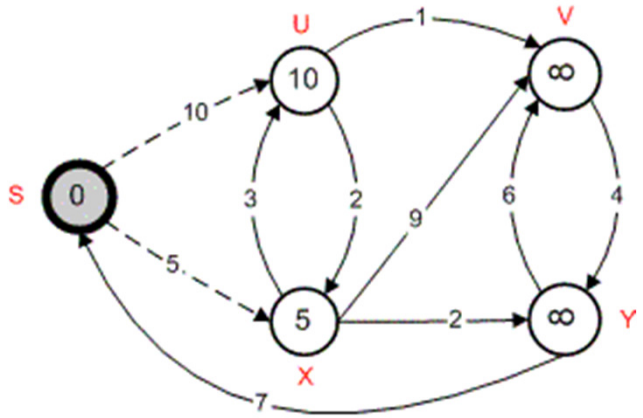
Algoritmo de Dijkstra



Vértice	Visitado	Distância	Caminho
s	sim	0	-
u	não	Infinito	-
x	não	Infinito	-
v	não	Infinito	-
y	não	Infinito	-

Defini-se inicialmente o nó de origem, neste caso *s*, e inclui-se este nó nos nós visitados. Atribui-se zero a sua distância porque o custo de ir de *s* a *s* é obviamente 0. Todos os outros nós tem as suas distâncias inicializadas com um valor bastante grande.

Algoritmo de Dijkstra

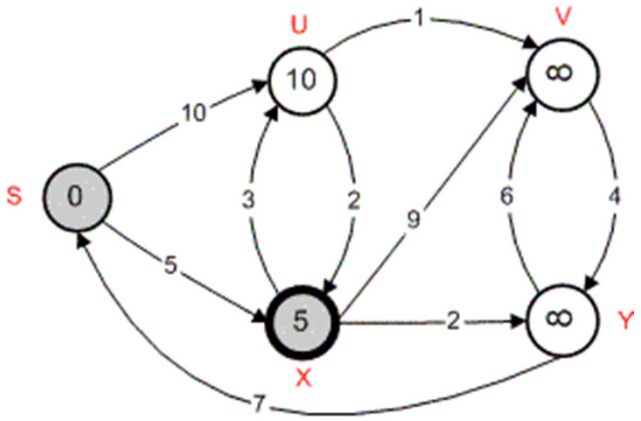


Vértice	Visitado	Distância	Caminho
s	sim	0	-
u	não	10	s
x	não	5	s
v	não	Infinito	-
y	não	Infinito	-

A partir de *s* consulta-se os vértices adjacentes a ele, que no grafo são *u* e *x*. Para todos os vértices adjacentes, que chamaremos *z*, calcula-se:

```
Se dist[z] > dist[s] + peso(s, z)
    dist[z] = dist[s] + peso(s, z)
    path[z] = s
Fim Se
```

Algoritmo de Dijkstra

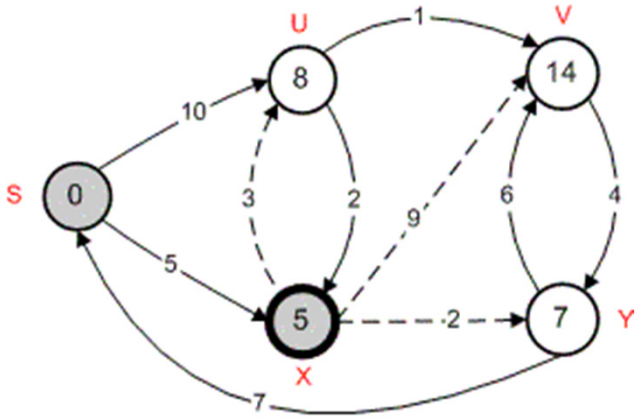


Vértice	Visitado	Distância	Caminho
s	sim	0	-
u	não	10	s
x	não	5	s
v	não	Infinito	-
y	não	Infinito	-

Dentre todos os vértices que não foram visitados escolhe-se aquele com a menor distância.

Neste caso é o vértice *x*, pois dist[x] = 5.

Algoritmo de Dijkstra

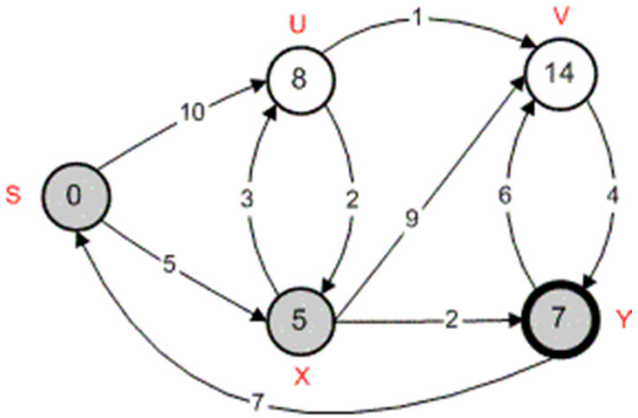


Vértice	Visitado	Distância	Caminho
s	sim	0	-
u	não	8	x
x	sim	5	s
v	não	14	x
y	não	7	x

Então, inclui-se *x nos nós visitados* e a partir de *x* consulta-se os vértices adjacentes a ele que não foram visitados, que no grafo são *u*, *v* e *y*. Para todos os vértices adjacentes, que chamaremos *z*, calcula-se:

```
Se dist[z] > dist[x] + peso(x, z)
      dist[z] = dist[x] + peso(x, z)
      path[z] = x
Fim Se
```

Algoritmo de Dijkstra

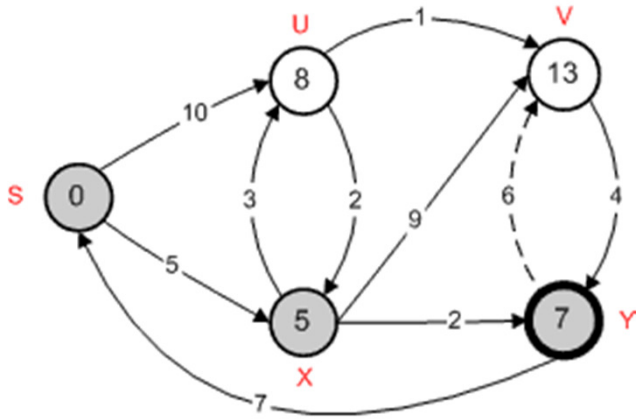


Vértice	Visitado	Distância	Caminho
s	sim	0	-
u	não	8	x
x	sim	5	s
v	não	14	x
y	não	7	x

Dentre todos os vértices que não foram visitados escolhe-se aquele com a menor distância.

Neste caso é o vértice *y*, pois dist[y] = 7.

Algoritmo de Dijkstra

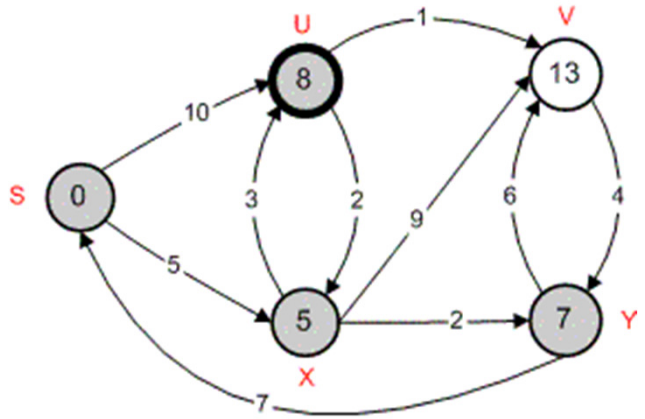


Vértice	Visitado	Distância	Caminho
s	sim	0	-
u	não	8	x
x	sim	5	s
v	não	13	y
y	sim	7	x

Inclui-se então *y* nos nós visitados e a partir de *y* consulta-se os vértices adjacentes a ele que não foram visitados, que no grafo é apenas o vértice *v*.

```
Se dist[v] > dist[y] + peso(y, v)
      dist[v] = dist[y] + peso(y, v)
      path[v] = y
Fim Se
```

Algoritmo de Dijkstra

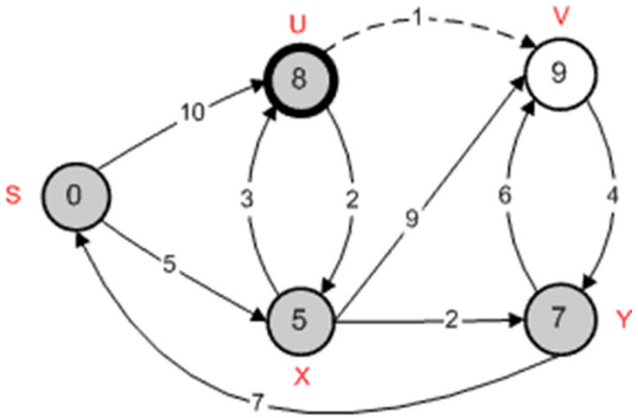


Vértice	Visitado	Distância	Caminho
s	sim	0	-
u	não	8	x
x	sim	5	s
v	não	13	y
y	sim	7	x

Dentre todos os vértices que não foram visitados escolhe-se aquele com a menor distância.

Neste caso é o vértice *u*, pois dist[u] = 8.

Algoritmo de Dijkstra

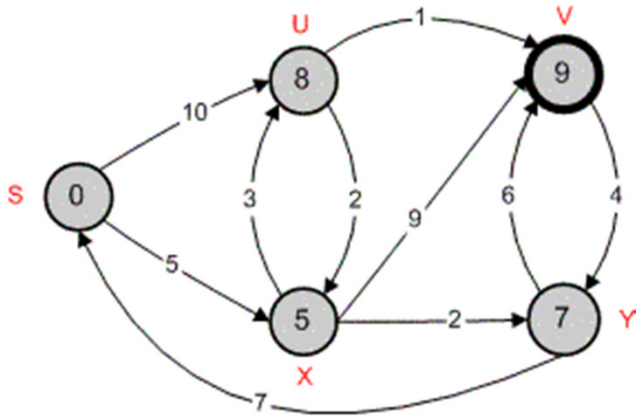


Vértice	Visitado	Distância	Caminho
s	sim	0	-
u	sim	8	x
x	sim	5	s
v	não	9	u
y	sim	7	x

Inclui-se então *u* nos nós visitados e a partir de *u* consulta-se os vértices adjacentes a ele que não foram visitados, que no grafo é apenas o vértice *v*.

```
Se dist[v] > dist[u] + peso(u, v)
      dist[v] = dist[u] + peso(u, v)
      path[v] = u
Fim Se
```

Algoritmo de Dijkstra

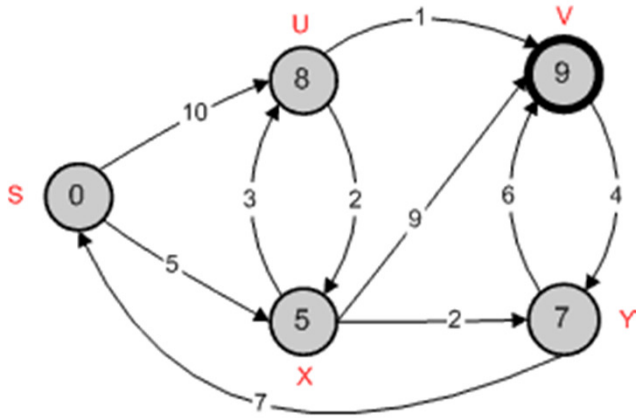


Vértice	Visitado	Distância	Caminho
s	sim	0	-
u	sim	8	x
x	sim	5	s
v	não	9	u
y	sim	7	x

Dentre todos os vértices que não foram visitados escolhe-se aquele com a menor distância.

Neste caso é o único vértice restante *v* e dist[v] = 9.

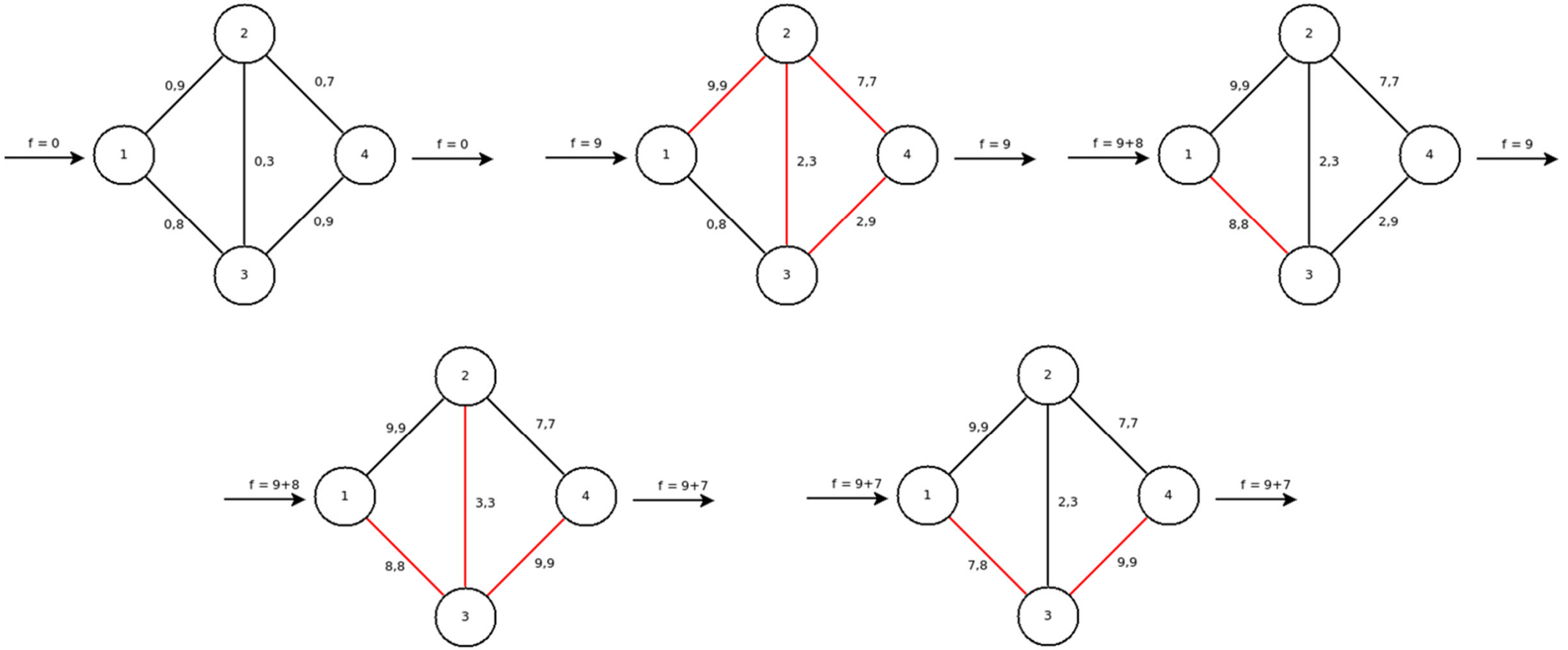
Algoritmo de Dijkstra



Vértice	Visitado	Distância	Caminho
s	sim	0	-
u	sim	8	x
x	sim	5	s
v	sim	9	u
y	sim	7	x

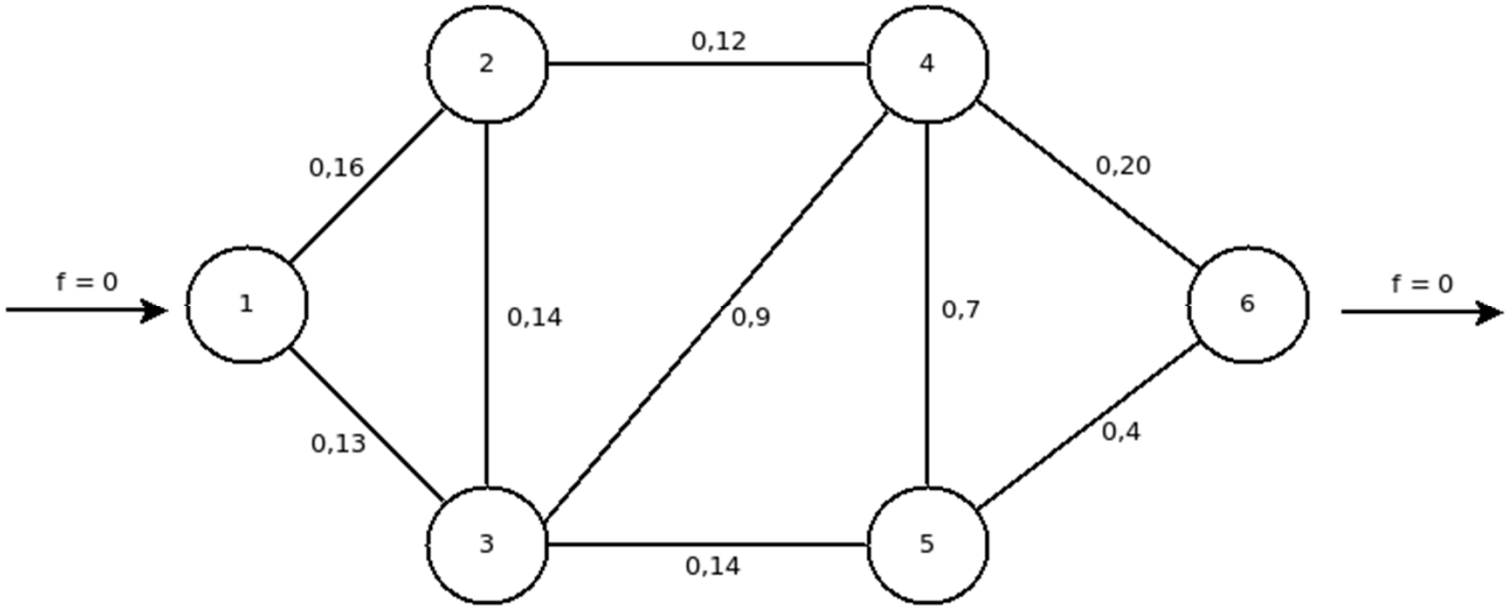
Por fim faz-se *v* pertencer aos nós visitados. Neste ponto, todos os vértices já foram visitados e a busca é finalizada

Algoritmo do Fluxo Maximo (versão Gulosa)

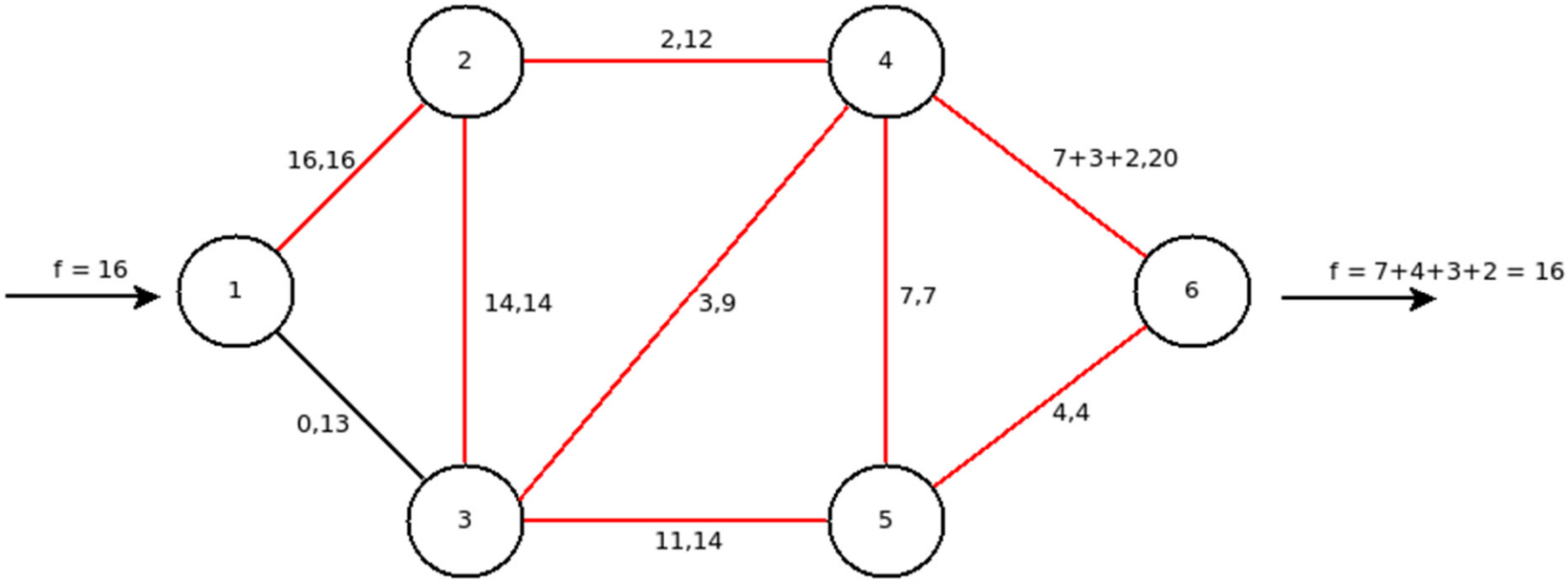


Vai sempre pelo ramo com maior capacidade

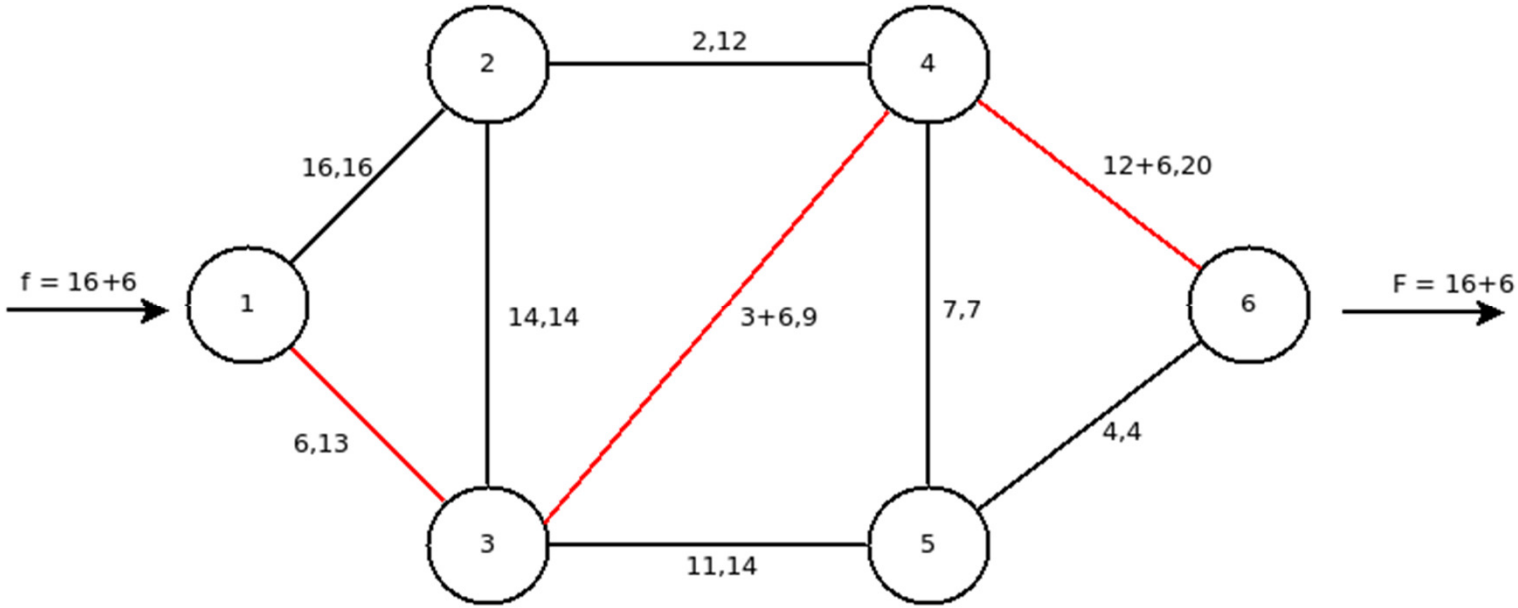
Algoritmo do Fluxo Maximo (versão Gulosa)



Algoritmo do Fluxo Maximo (versão Gulosa))

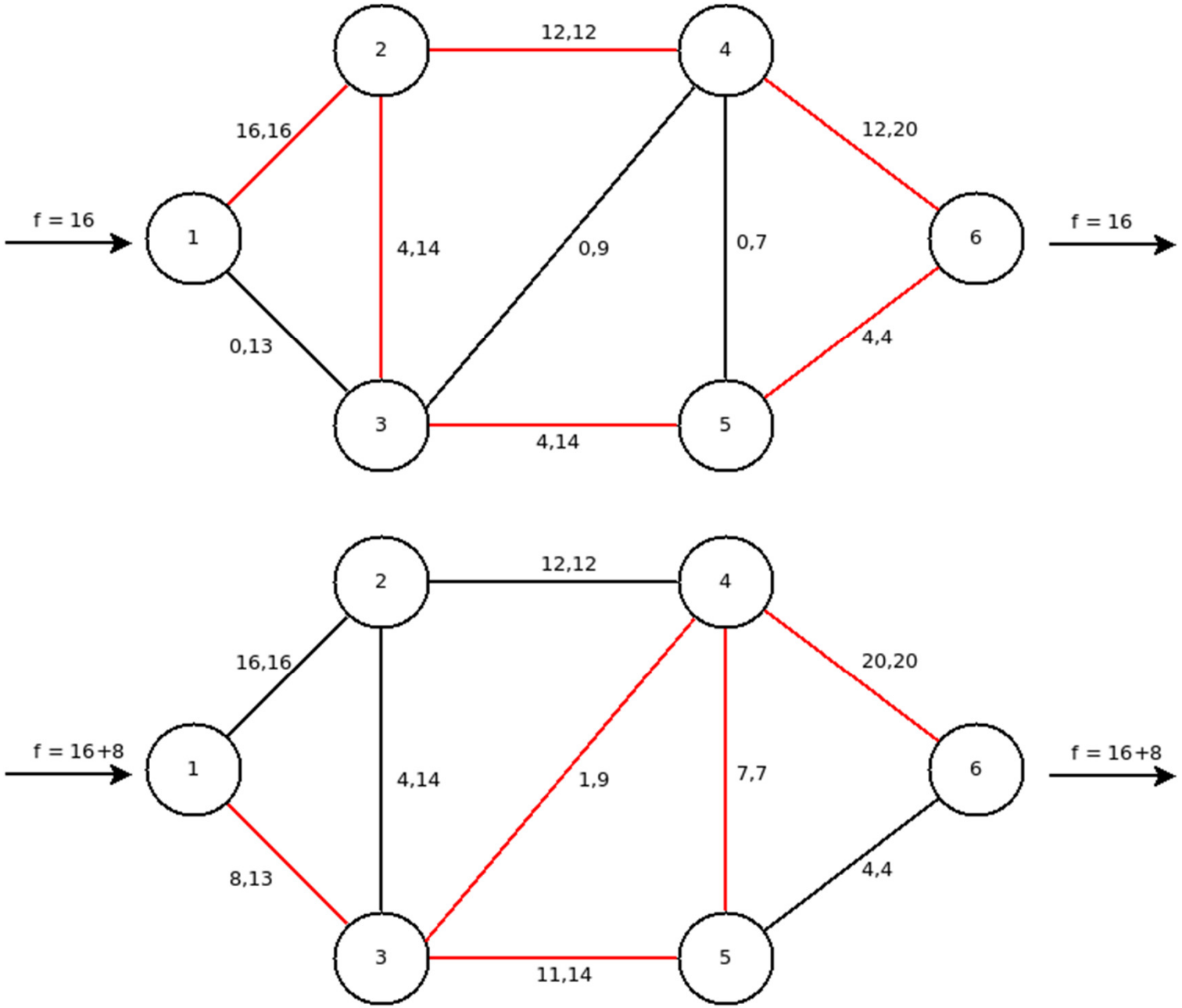


Algoritmo do Fluxo Maximo (versão Gulosa)

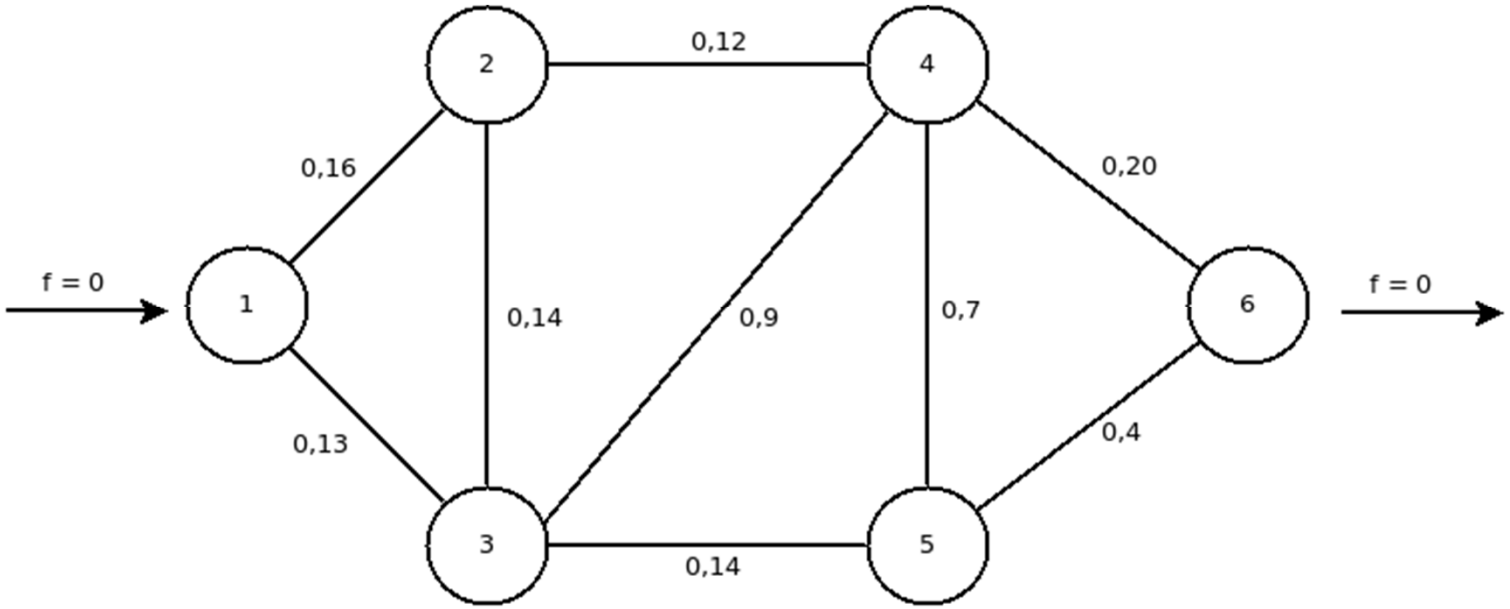


Será que encontra a solução óptima?

A versão Gulosa não encontra a solução optima

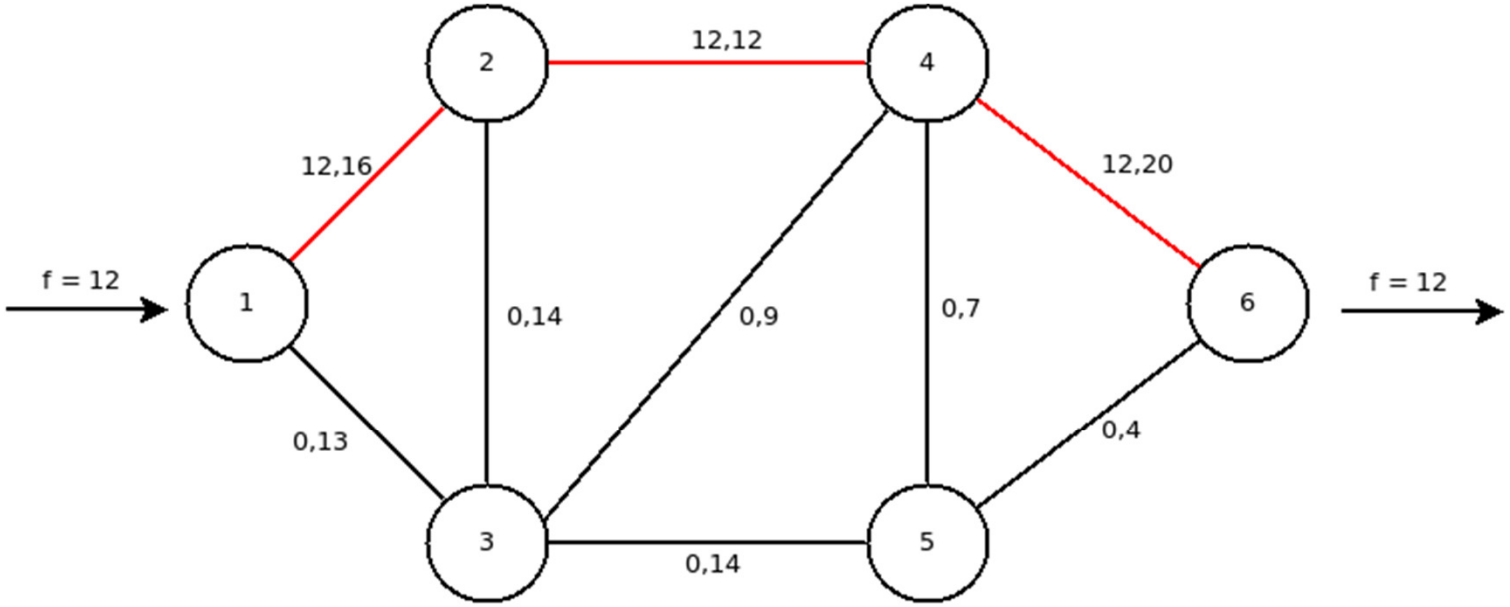


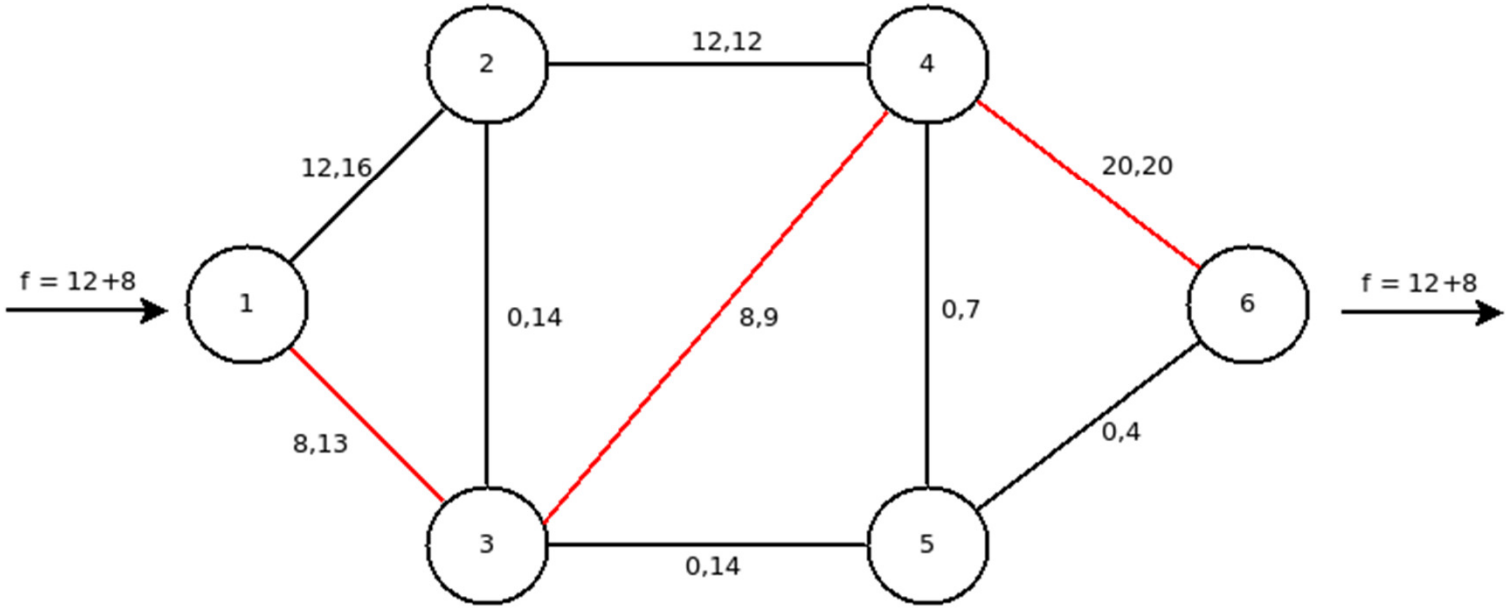
Algortimo de Ford-Fulkerson



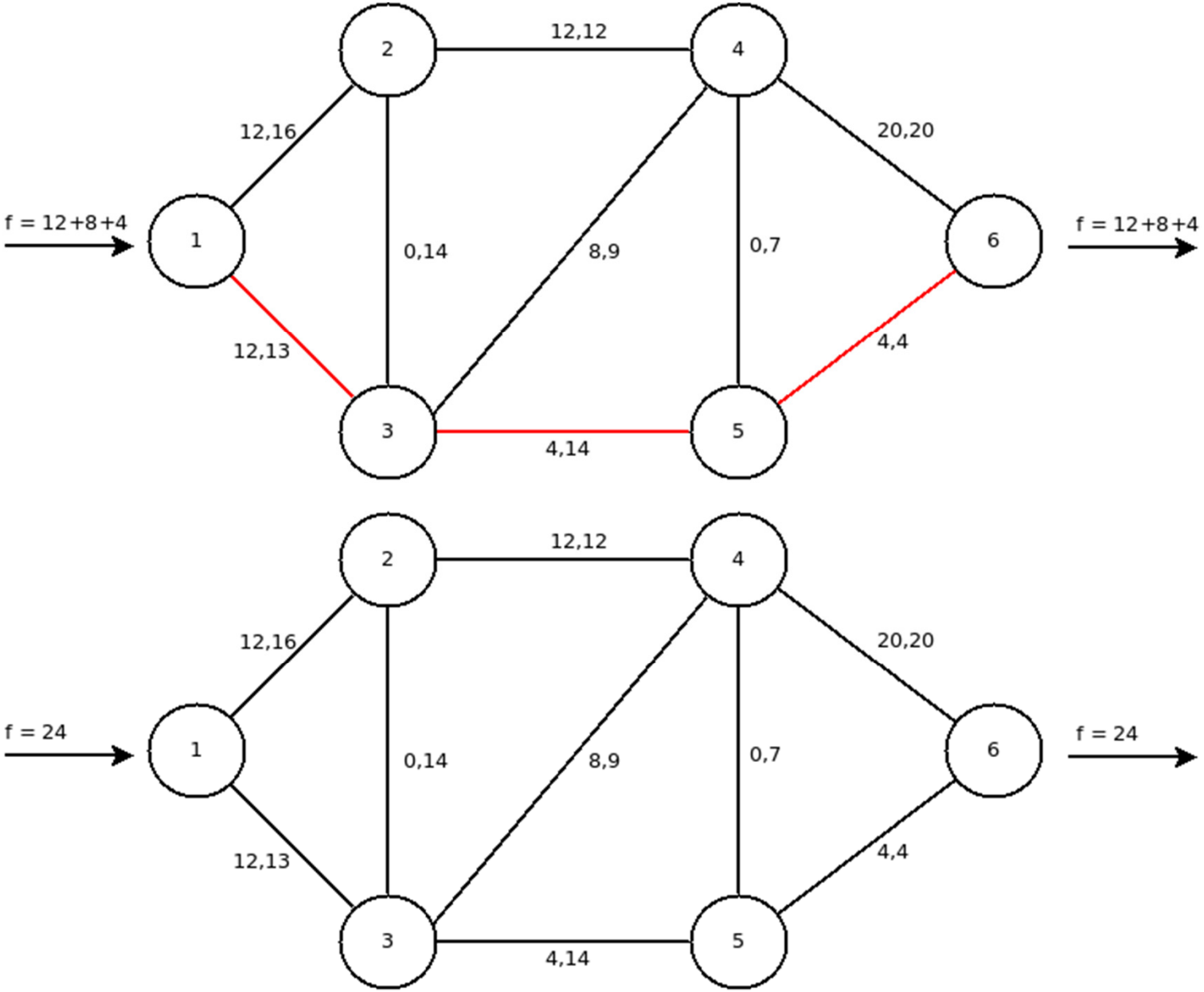
Escolher sempre o caminho que permite um fluxo máximo

Algortimo de Ford-Fulkerson

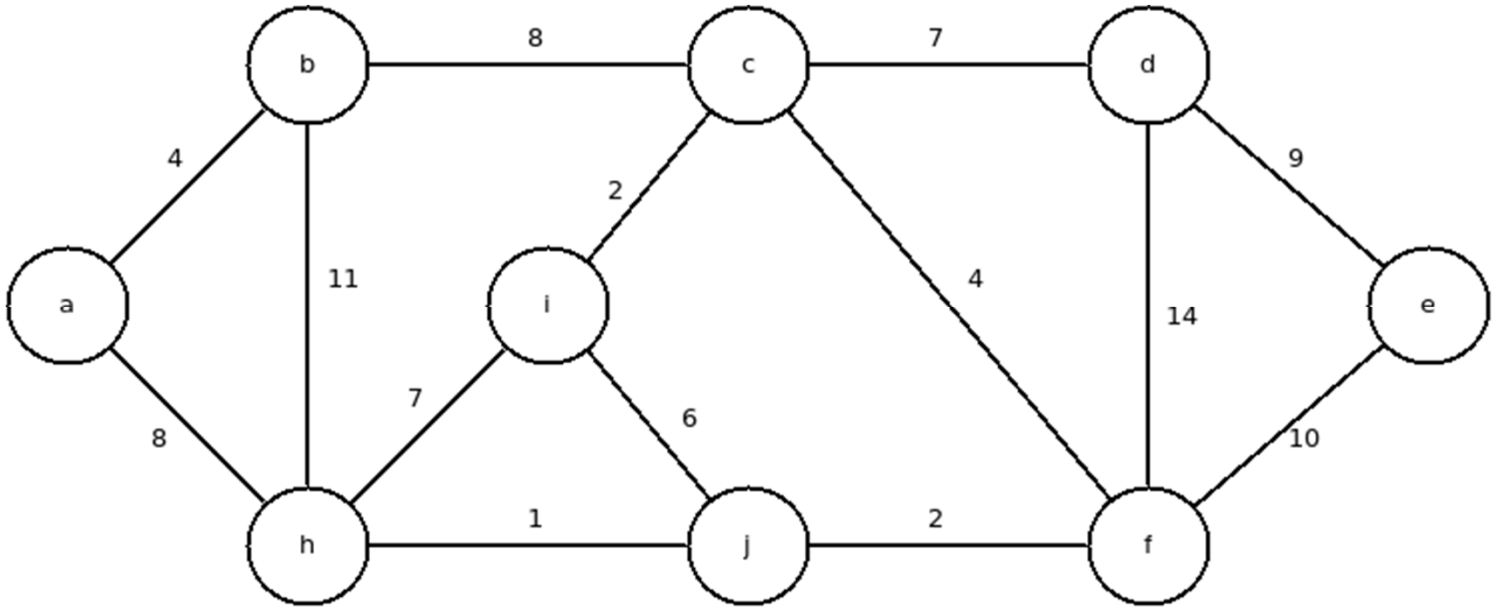




Algortimo de Ford-Fulkerson

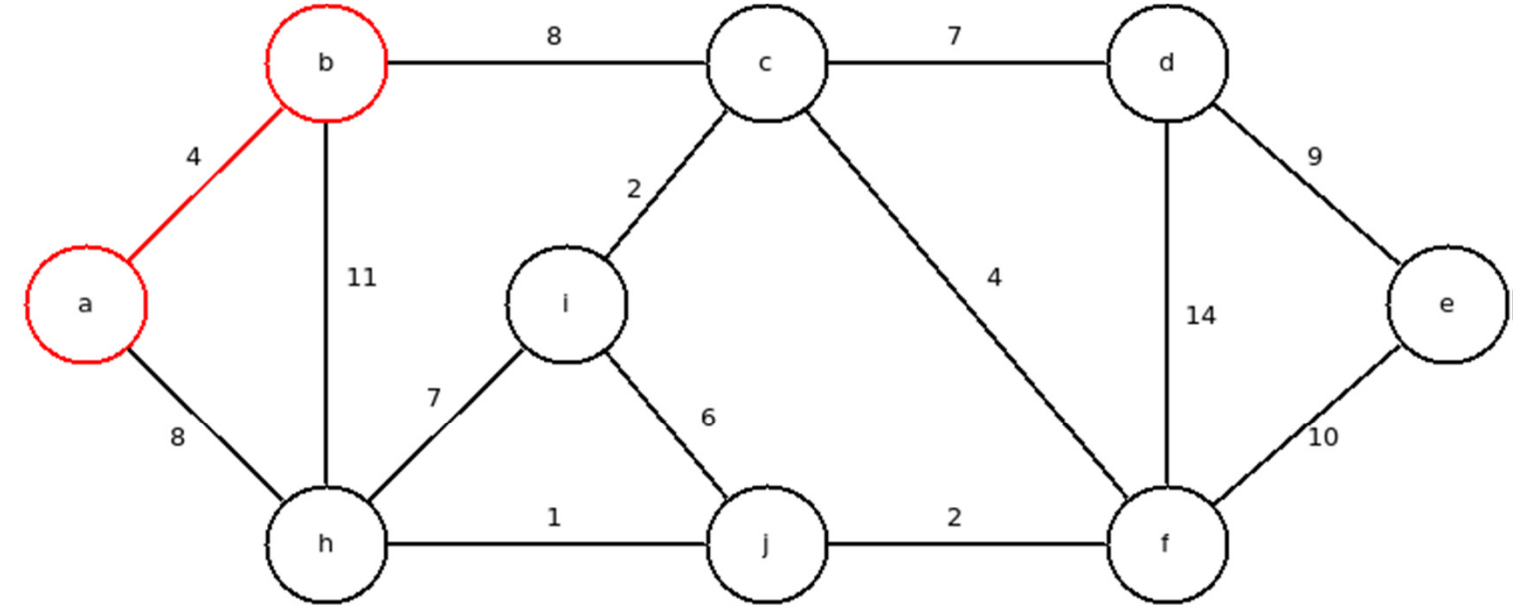


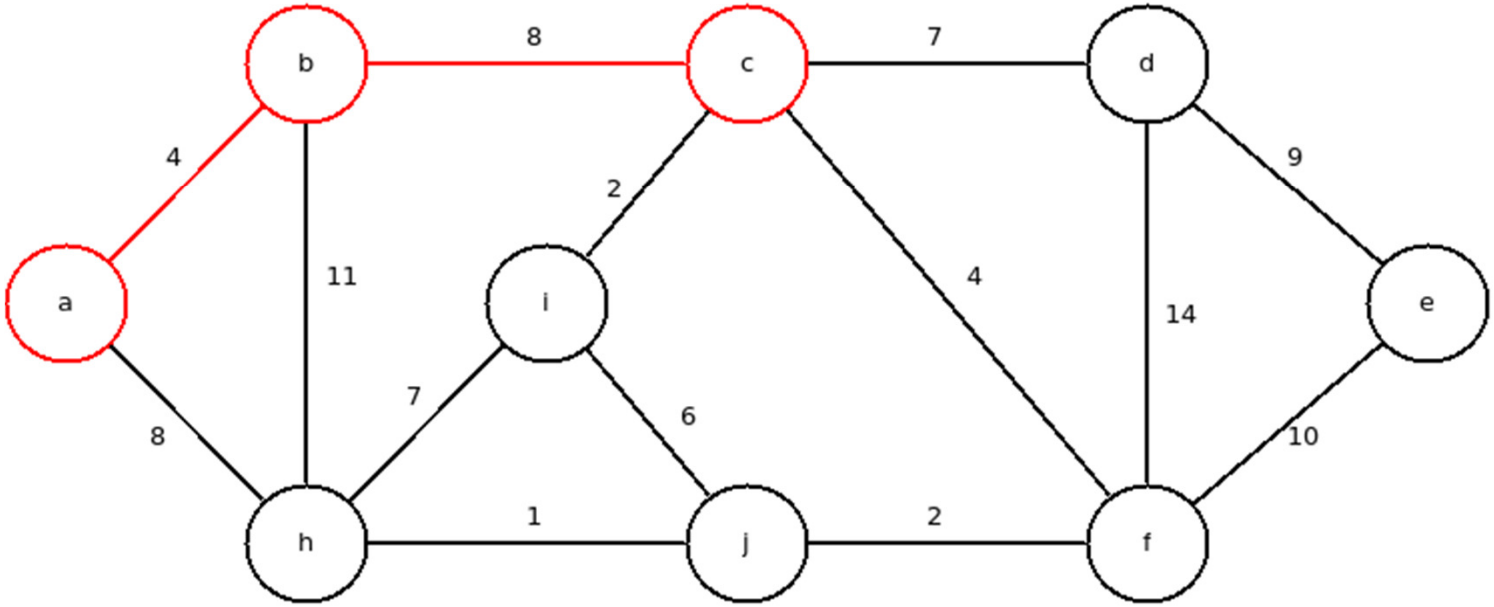
Algortimo de Primm



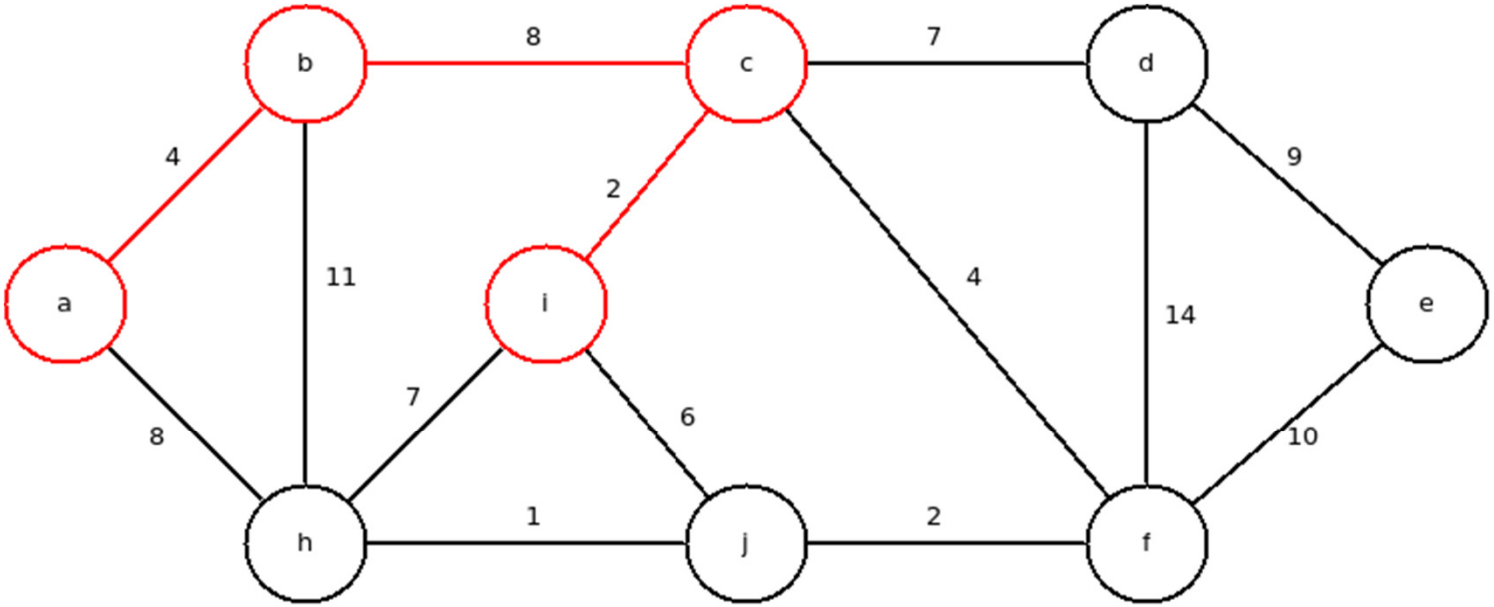
Partindo de um nó inical, escolhe-se sempre a aresta com menor peso, excepto quando a aresta fecha o caminho

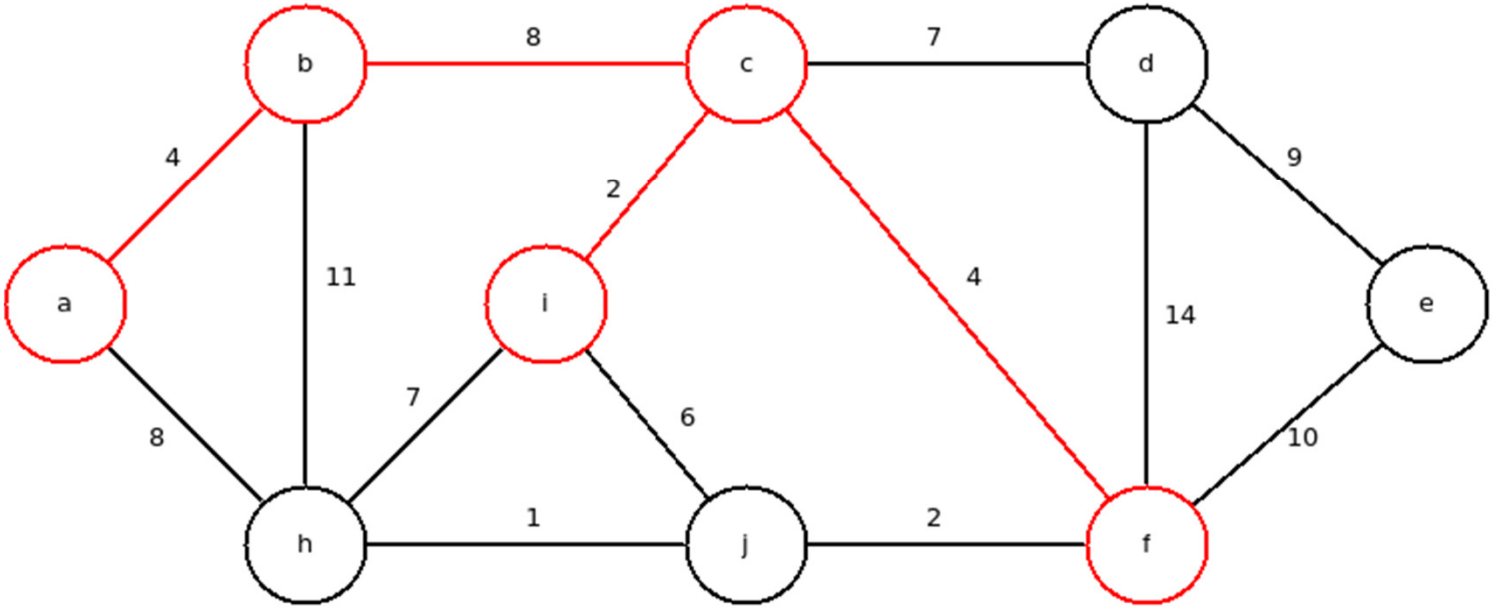
Algortimo de Primm



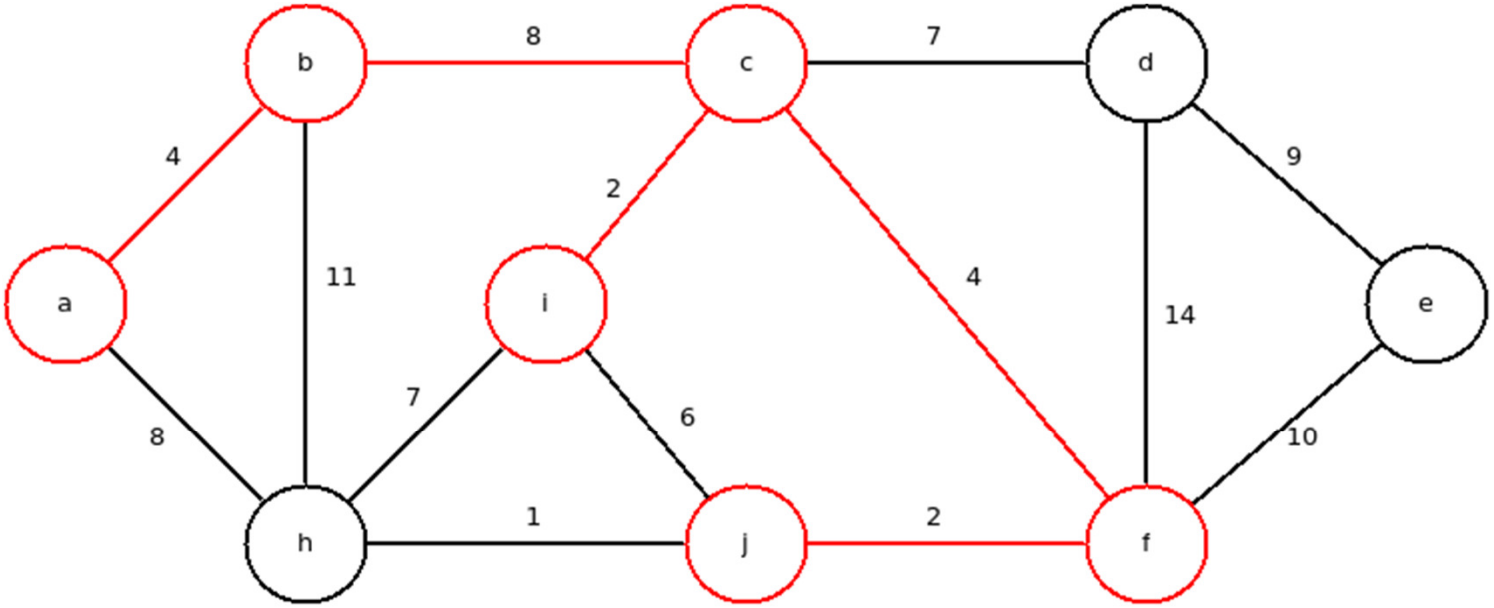


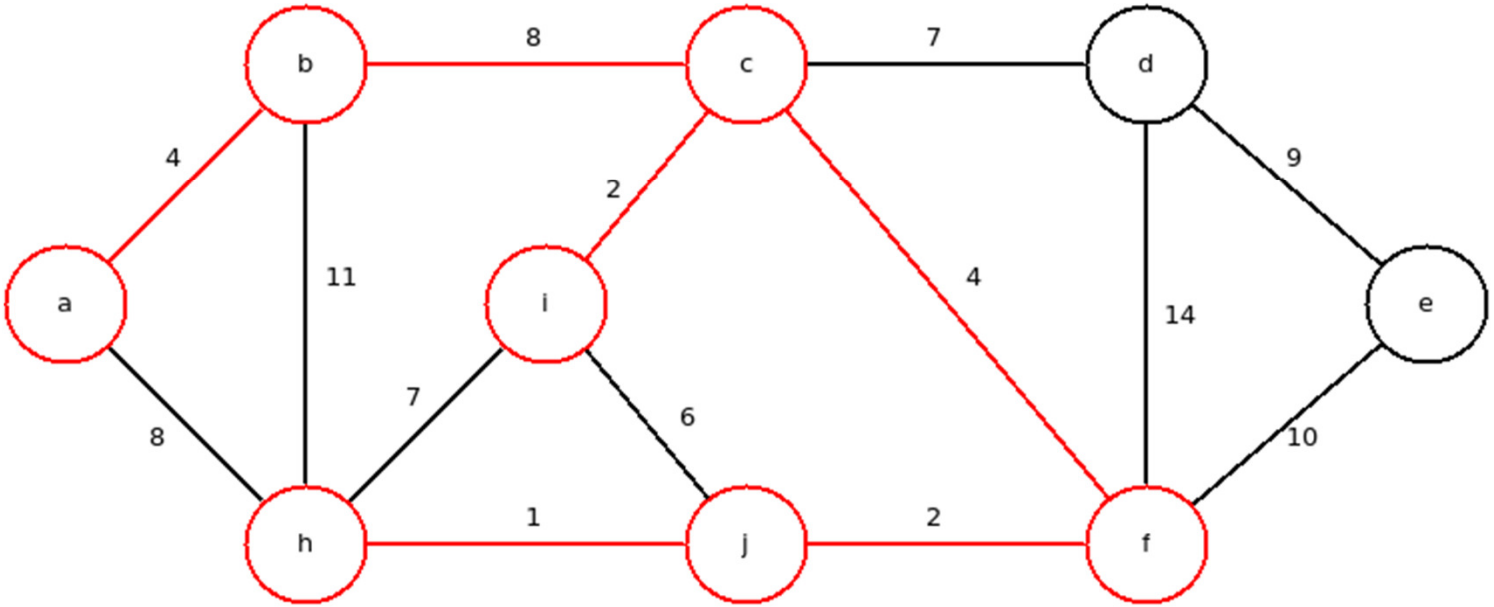
Algortimo de Primm



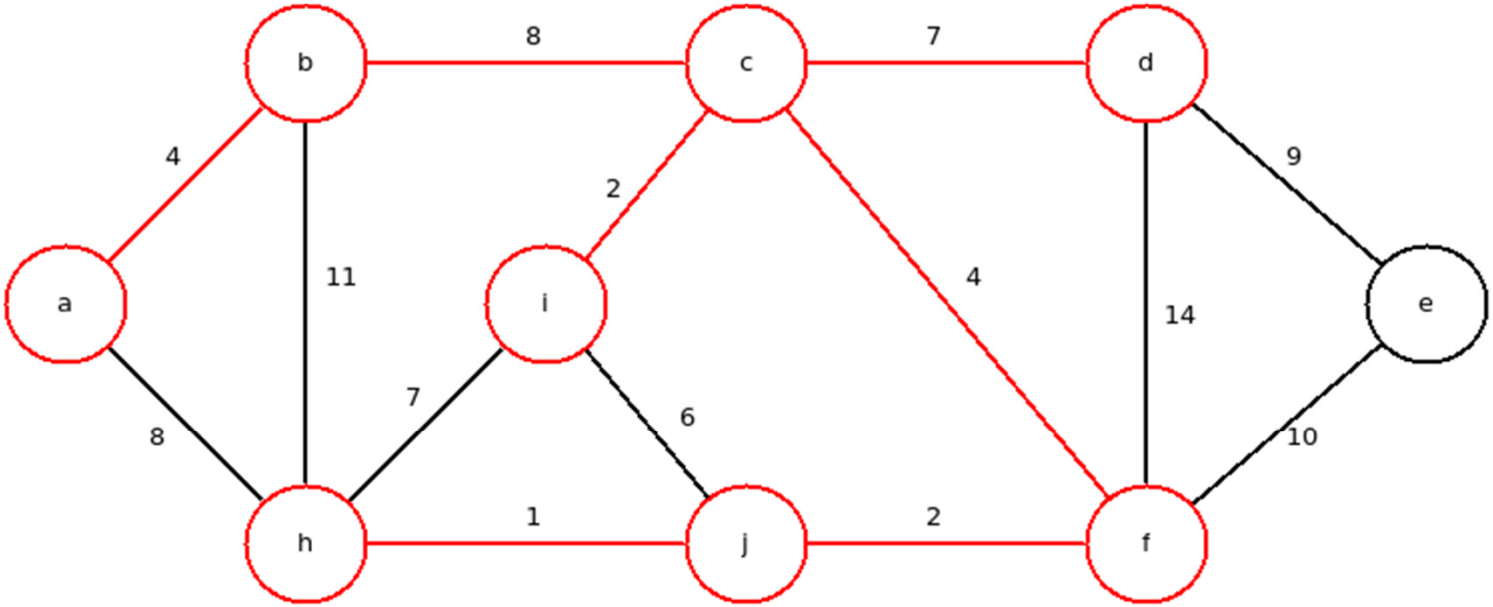


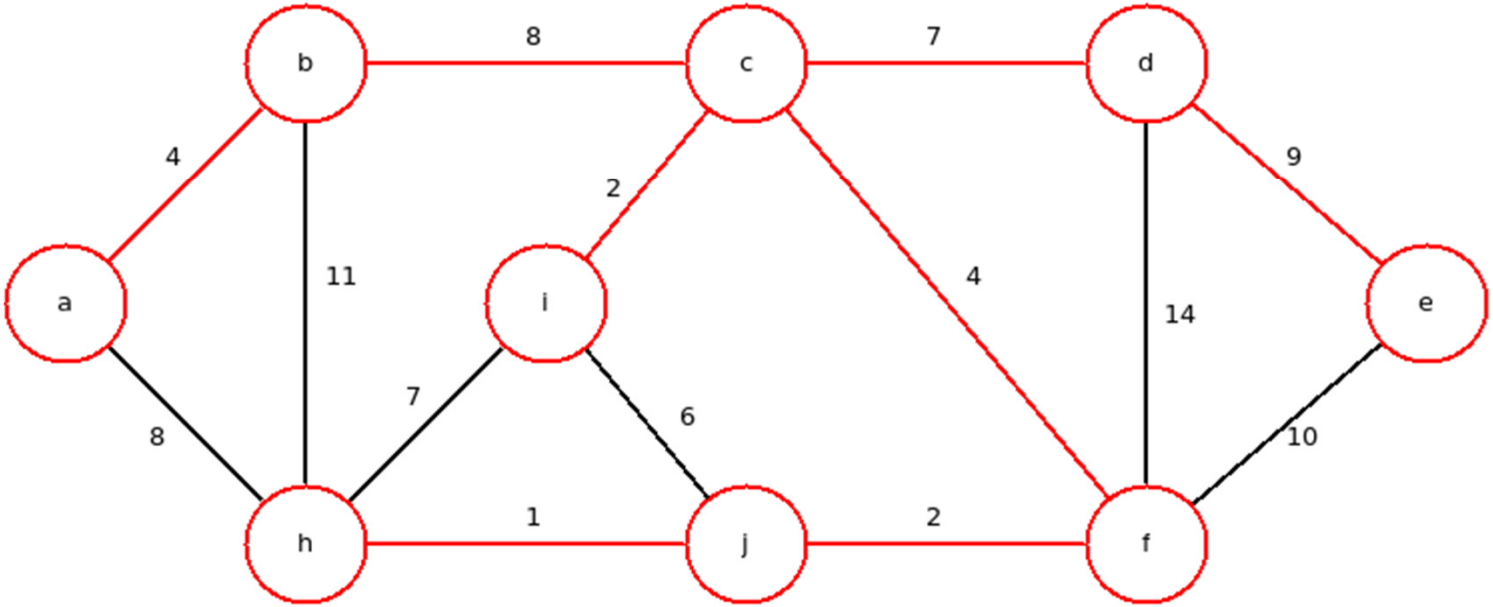
Algortimo de Primm



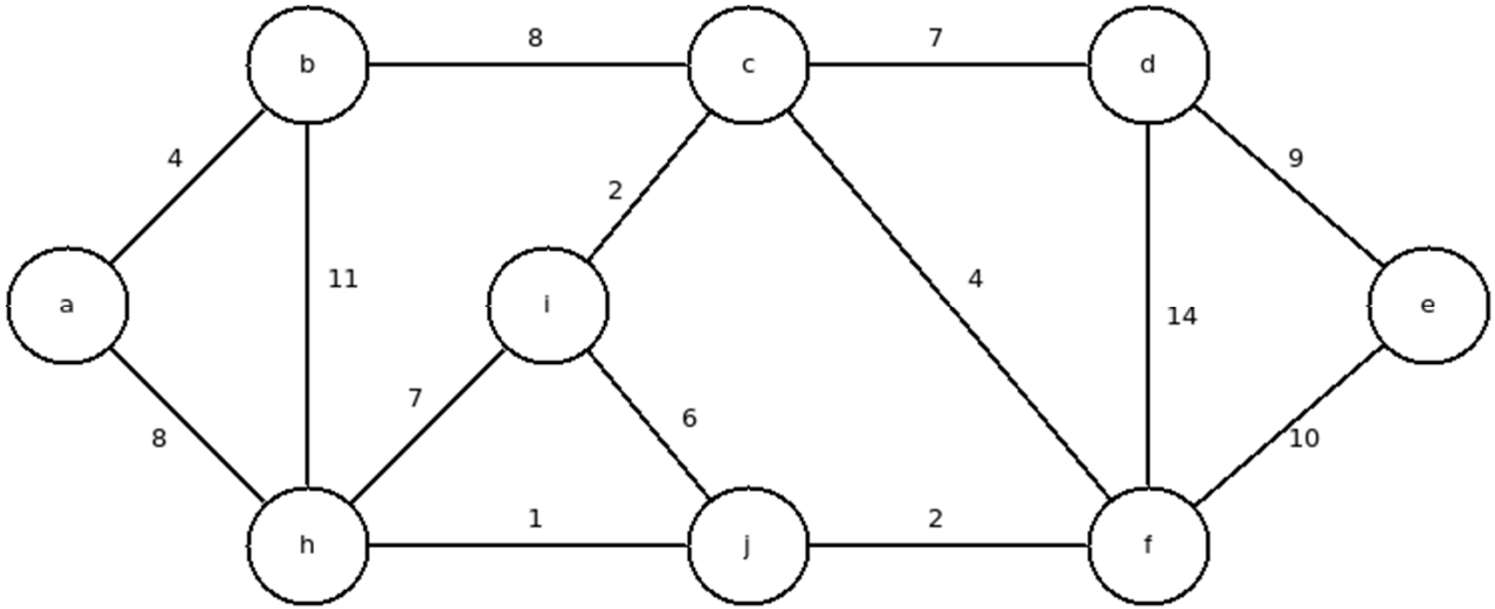


Algortimo de Primm



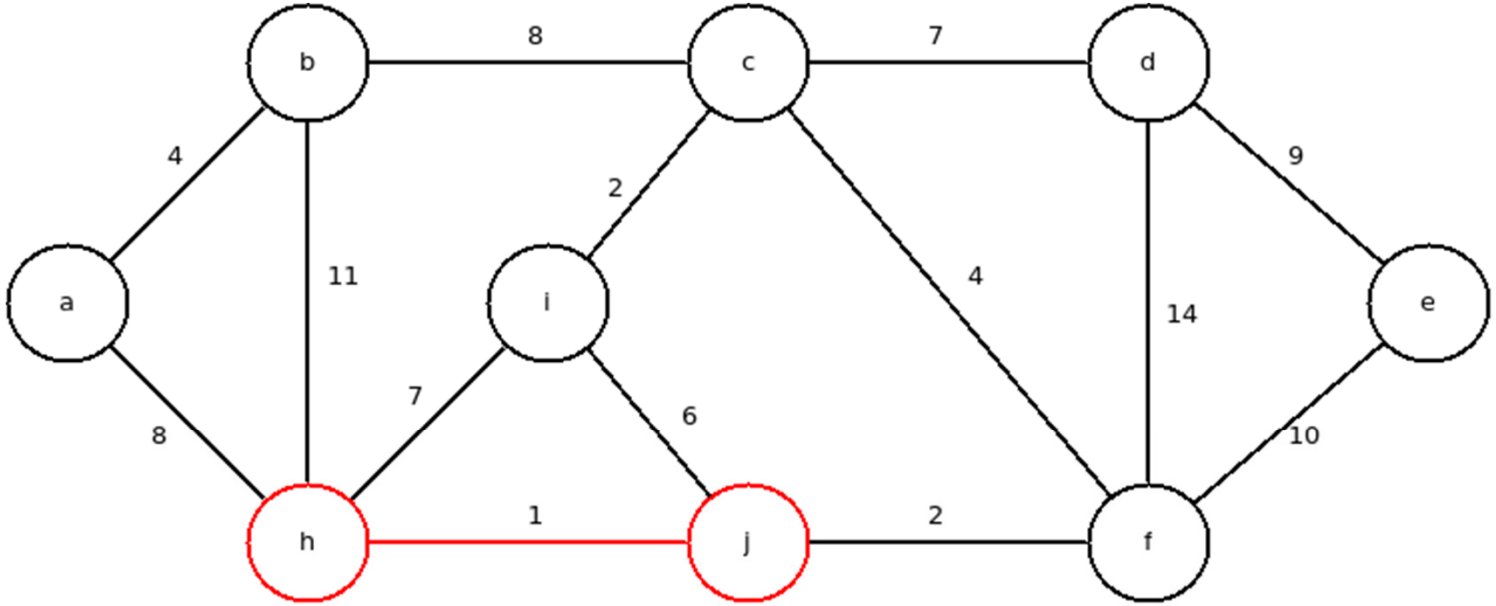


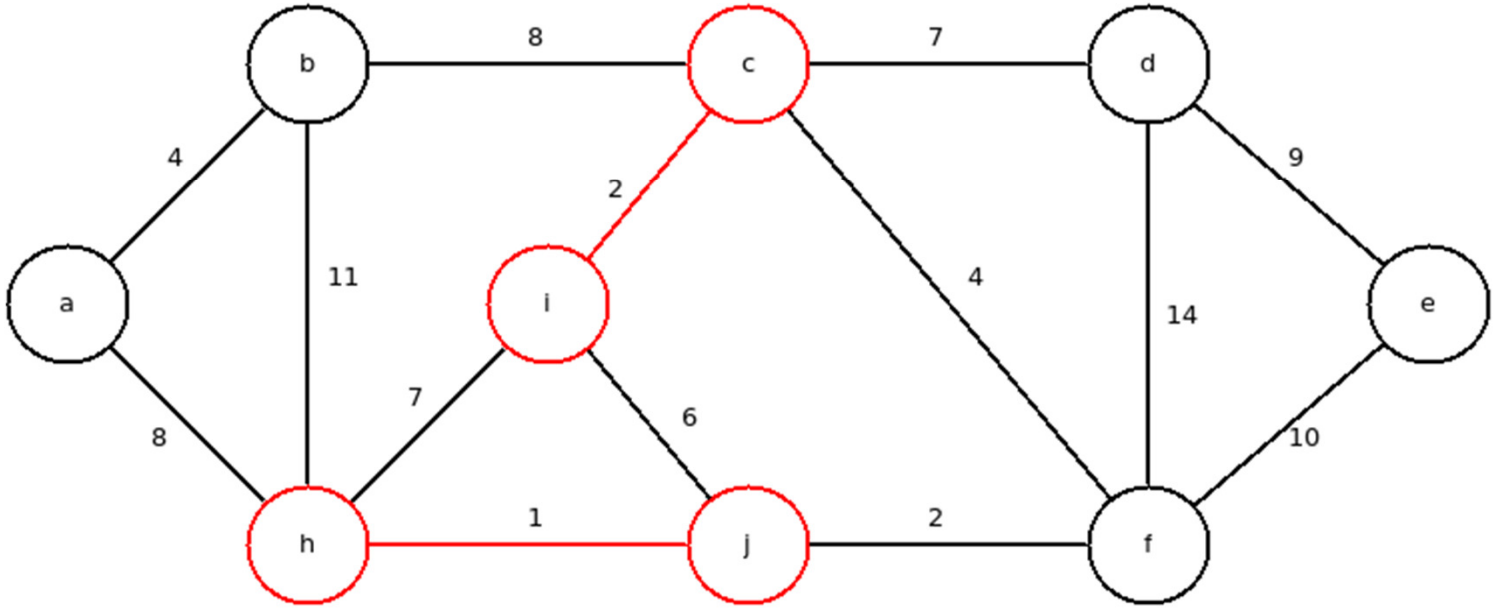
Algortimo de Kruskal



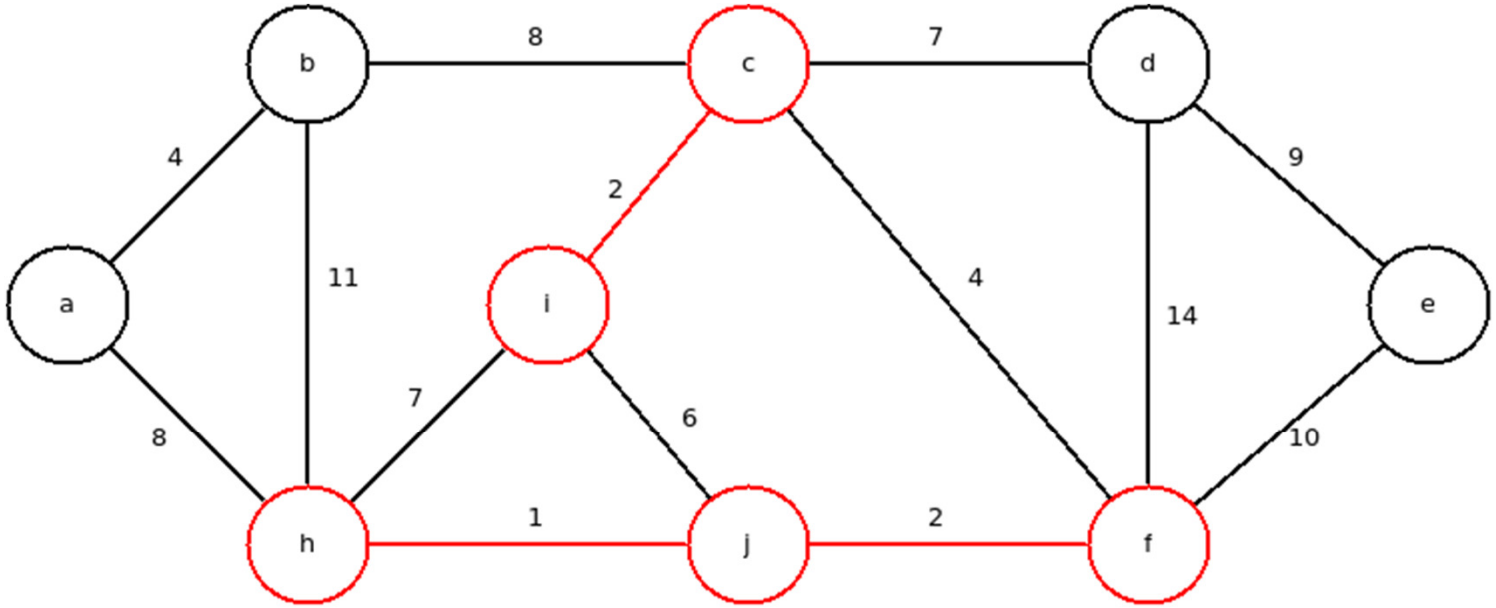
Escolhe sempre a aresta com menos peso, excepto quando a aresta fecha o caminho

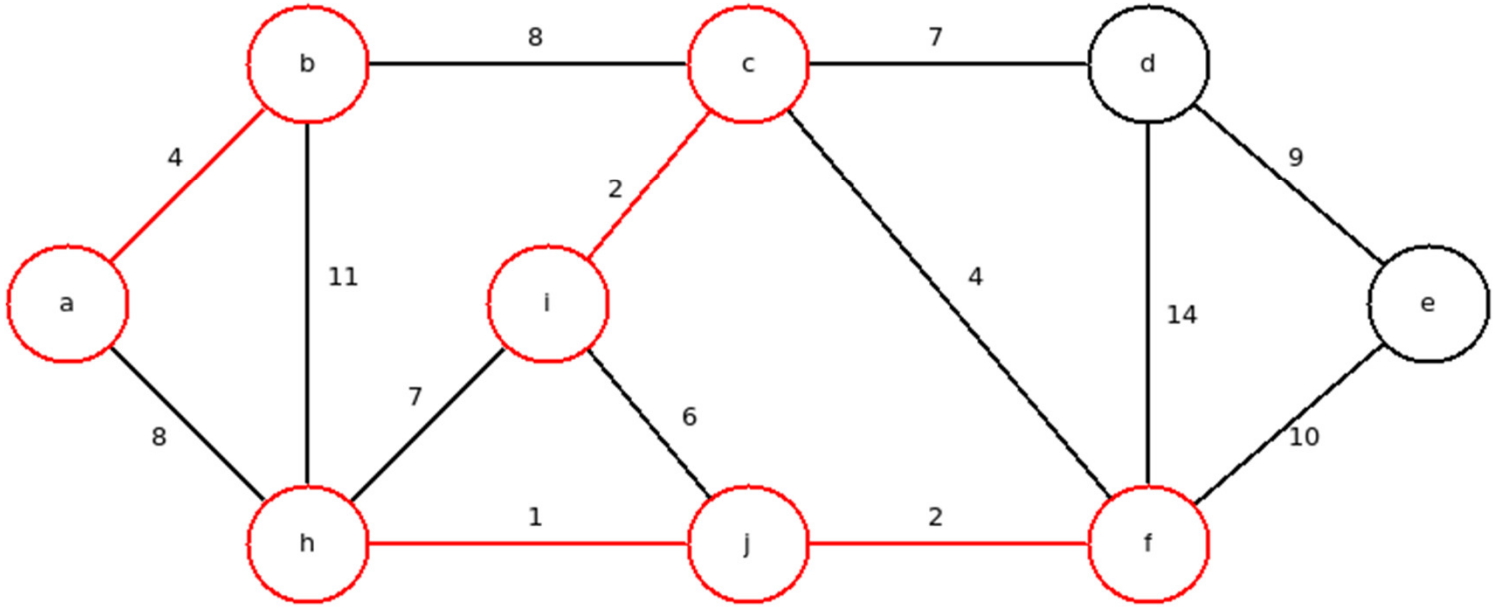
Algortimo de Kruskal



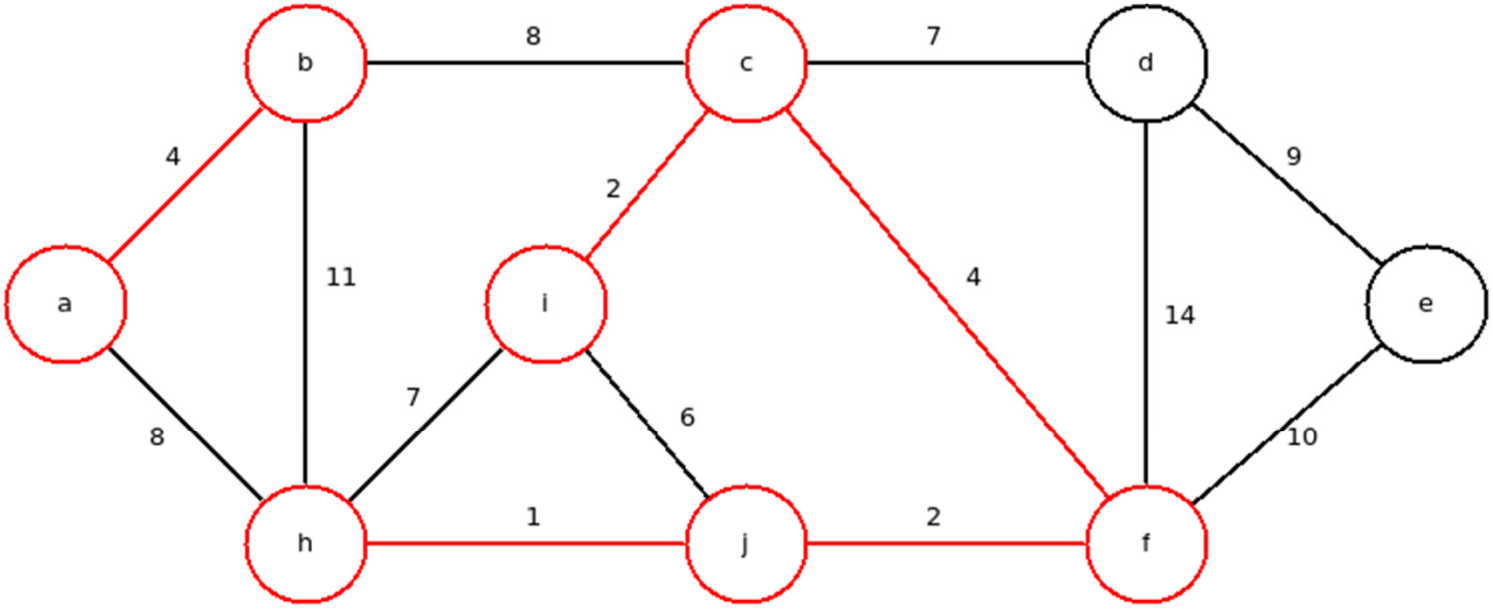


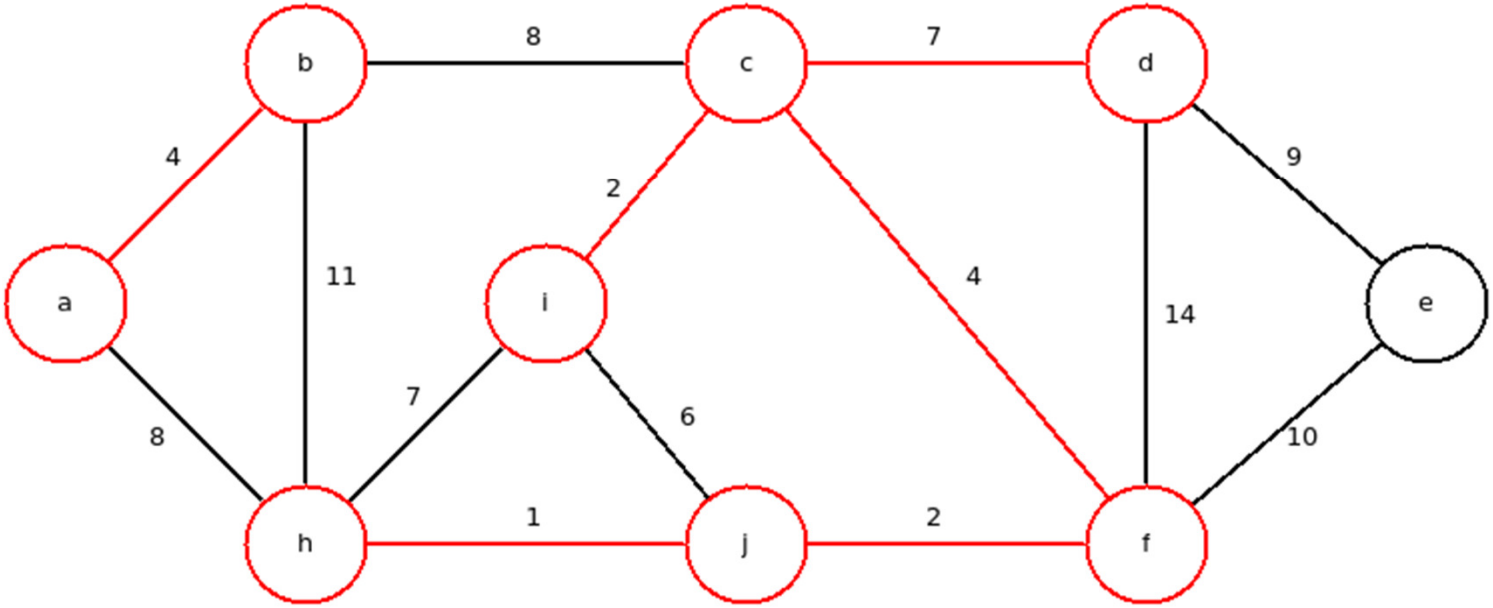
Algortimo de Kruskal





Algortimo de Kruskal





Algortimo de Kruskal

